MyEmulator2 Assembly Language Programming Manual

REM project

Revision 1.0 — 2026-09-22

Document status and scope

This manual describes the current **MyEmulator2** 32-bit target implemented by the tracked QEMU target, the `linux/arch/myemulator2` port, the project binutils/GAS port, the GCC backend, and the MyEmulator2 musl/minilibc work. It is an implementation manual, not a promise that every planned ABI feature is complete.

The repository also contains a legacy **MyEmulator** 8-bit-data/16-bit-address firmware target. Its instruction set is intentionally different and is documented separately in `docs/instruction-set.md`, `docs/architecture.md`, and `docs/assembler.md`. Do not combine examples from those documents with the MyEmulator2 examples in this manual.

The current implementation is authoritative where older design prose differs. Features explicitly labelled *planned*, *candidate*, or *not implemented* must not be used as if they were available.

Verified implementation boundaries

Area	Current status
32-bit fixed-width CPU and MMU	Implemented in QEMU and covered by CPU/system tests
ELF32 little-endian object and static executable format	Implemented project ABI
GNU assembler and linker overlay	Implemented for the project target
GCC integer backend and soft-float support	Implemented and exercised by toolchain tests
Linux entry, user ELF loading, process lifecycle	Implemented and tested
Basic Linux syscall dispatcher	Implemented; table is intentionally incomplete
Serial console, timer, and ext4 block device	Implemented in the current Linux machine
Full signals, dynamic linking, complete libc, stable userspace ABI	Not complete

1. Programmer-visible architecture

MyEmulator2 is a little-endian, byte-addressed, 32-bit processor. General integer values, virtual addresses, physical addresses, and instruction words are 32 bits. Arithmetic wraps modulo (2^{32}) unless an instruction raises an exception.

Instructions are fixed-width 32-bit words, stored little-endian, and must be 4-byte aligned. Normal sequential execution increments `PC` by four.

1.1 General registers

There are sixteen encoded general-purpose registers. Register-field values 16–31 are reserved and are rejected by the decoder.

Register	Alias	Meaning	ABI treatment
r0	zero	Reads as zero; writes are discarded	Constant
r1–r4	—	General values and first four arguments	Caller-saved
r5–r12	—	General values	Callee-saved
r13	sp	Banked stack pointer	Callee-managed
r14	lr	Link/return register	Preserved when a function calls
r15	—	General temporary; GCC uses it as a frame-base register in relevant builds	Caller-saved

PC is a separate architectural register. SR is a separate status/control register. r13 selects USP in user mode and SSP in supervisor mode.

1.2 Status register and privilege

SR bit assignments are:

Bits	Name	Meaning
0	CF	Carry for addition; set when an unsigned subtraction borrows
1	OF	Signed two's-complement overflow
2–4	IPL	Interrupt priority level, 0–7
5	S	Supervisor mode when set
6–31	reserved	Read as zero; writes have no effect

There are two execution modes. User mode cannot directly access supervisor state. Supervisor code can select the user and supervisor stack pointers through system-register operations. An interrupt is accepted only when its level is greater than IPL; IPL=7 masks ordinary IRQs.

The current Linux port relies on the system registers SR, USP, SSP, VBR, PTBR, MMCR, TIME_LO, TIME_HI, TIMECMP_LO, TIMECMP_HI, TP, and DFSP. A 32-bit transfer never claims to read or write the 64-bit timer as one operation.

1.3 Exceptions and interrupts

The current vector assignments are:

Vector	Cause
0	Illegal instruction
1	Privilege violation
2	Instruction alignment
3	Data alignment
4–5	Instruction page-not-present/protection
6–9	Load/store page-not-present/protection
10	Divide by zero
11	Arithmetic overflow

Vector	Cause
12	System call
13	Breakpoint
14	NMI
15	Double fault
16–22	IRQ1–IRQ7

The CPU builds an exception frame on the supervisor stack. Linux expands this into its software `pt_regs` frame before calling C code. General registers are not automatically preserved by the hardware; handlers must save registers they modify.

IRQ entry saves the next instruction address and complete pre-interrupt status. `RFE`/the Linux return path restores the saved state. Nested interrupts are possible only for higher-priority levels.

1.4 Memory and alignment

The virtual and physical address spaces are 32-bit. Loads and stores use a base register plus a signed 13-bit displacement. `LB`/`LBU` access one byte, `LH`/`LHU` two bytes, and `LW` four bytes. `SB`, `SH`, and `SW` store one, two, and four bytes respectively. Natural alignment is required for multi-byte accesses; misalignment raises the data-alignment exception.

The current development machine loads `-kernel` bytes at physical address zero. The Linux user linker script places user images near `0x02000000` and the kernel port maintains the required mappings. These are distinct from the legacy 16-bit firmware map.

2. Instruction encoding

Every instruction occupies one 32-bit little-endian word. Bit 31 is the most significant bit of the decoded word.

```
R:       op[31:26] rd[25:21] ra[20:16] rb[15:11] fn[10:6] reserved[5:0]
I:       op[31:26] rd[25:21] ra[20:16] sub[15:12] imm[11:0]
MEM:     op[31:26] rt[25:21] base[20:16] size[15:13] disp[12:0]
B:       op[31:26] cond[25:23] ra[22:18] rb[17:13] disp[12:0]
J:       op[31:26] disp[25:0]
IND:     op[31:26] link[25] ra[24:20] reserved[19:0]
U:       op[31:26] rd[25:21] imm[20:1] reserved[0]
SYS:     op[31:26] sysop[25:22] reg[21:17] sysreg[16:11] reserved[10:0]
SYSCALL: op[31:26] immediate[25:0]
TLB:     op[31:26] page[25] ra[24:20] reserved[19:0]
CAS:     op[31:26] rd[25:21] rs[20:16] ra[15:11] reserved[10:0]
```

Primary opcode allocation:

Opcode	Family
`0x00`	R-format ALU and comparisons
`0x01`	Immediate ALU and shifts
`0x02`	Loads
`0x03`	Stores
`0x04`	Conditional branches
`0x05`	`J`
`0x06`	`JAL`
`0x07`	`JR`/`JALR`
`0x08`	`LUI`
`0x09`	System-register operations
`0x0A`	`SYSCALL`

Opcode	Family
0x0B	TLB flush
0x0C	Compare-and-swap
0x0D–0x3F	Reserved

The machine-readable source is `docs/myemulator2-encoding.json`. Reserved fields must remain zero. A malformed or reserved encoding raises an illegal instruction exception.

3. Instruction reference

The following list is the complete instruction set represented by the current MyEmulator2 encoding manifest.

3.1 Integer and logical operations

Instruction	Semantics
`add rd, ra, rb`	`rd = ra + rb`
`adc rd, ra, rb`	Add with carry
`sub rd, ra, rb`	`rd = ra - rb`
`sbc rd, ra, rb`	Subtract with carry/borrow
`mul rd, ra, rb`	Low 32 bits of signed product
`mulh rd, ra, rb`	High 32 bits of signed product
`mulhu rd, ra, rb`	High 32 bits of unsigned product
`div rd, ra, rb`	Signed quotient; zero divisor traps
`divu rd, ra, rb`	Unsigned quotient; zero divisor traps
`rem rd, ra, rb`	Signed remainder; zero divisor traps
`remu rd, ra, rb`	Unsigned remainder; zero divisor traps
`and rd, ra, rb`	Bitwise AND
`or rd, ra, rb`	Bitwise OR
`xor rd, ra, rb`	Bitwise XOR
`not rd, ra, r0`	Bitwise complement; `rb` must be `r0`
`sll rd, ra, rb`	Logical left shift
`srl rd, ra, rb`	Logical right shift
`sra rd, ra, rb`	Arithmetic right shift
`rol rd, ra, rb`	Rotate left
`ror rd, ra, rb`	Rotate right
`seq rd, ra, rb`	`rd = (ra == rb)`
`sne rd, ra, rb`	`rd = (ra != rb)`
`slt rd, ra, rb`	Signed less-than
`sge rd, ra, rb`	Signed greater-than-or-equal
`sltu rd, ra, rb`	Unsigned less-than
`sgeu rd, ra, rb`	Unsigned greater-than-or-equal

Immediate suboperations are `addi`, `subi`, `andi`, `ori`, `xori`, `slli`, `srli`, and `srai`. Arithmetic immediates are sign-extended; logical immediates are zero-extended. Immediate shift counts use `imm[4:0]` and require the upper immediate bits to be zero.

Only `add`, `adc`, `sub`, `sbc`, `addi`, and `subi` update the defined carry/overflow state. Multiplication, division, remainder, logical, comparison, load/store, and control-transfer operations leave `CF/OF` unchanged. Logical and comparison operations do not provide a conventional N/Z flag register; comparison results are written to a register and the Linux/GCC code must not assume flags exist beyond the documented `SR` bits.

3.2 Loads and stores

```
lb   r1,  0(r5)       # sign-extended byte
lbu  r1,  1(r5)       # zero-extended byte
lh   r2,  2(r5)       # sign-extended halfword
lhu  r2,  4(r5)       # zero-extended halfword
lw   r3,  8(r5)       # 32-bit word
sb   r3,  0(r5)
sh   r3,  2(r5)
sw   r3,  4(r5)
```

The effective address is `(base + sign_extend(disp13)) mod 2^32`. There is no implicit base-register update. A load writes `rt`; a store reads `rt`.

3.3 Control transfer

`J` and `JAL` encode a signed 26-bit displacement in instruction words. Conditional branches encode a signed 13-bit displacement and compare two registers:

```
beq  r1, r2, equal
bne  r1, r2, different
blt  r1, r2, less
bge  r1, r2, not_less
bltu r1, r2, unsigned_less
bgeu r1, r2, unsigned_not_less
j    loop
jal  function
jr   r14
jalr r14, r5
```

For a control-transfer instruction at `P`, the target is:

```
target = (P + 4) + sign_extend(encoded_displacement) * 4
```

Targets must be 4-byte aligned. `JAL` writes the following instruction address to `r14` before transferring. `JR` transfers without linking; `JALR` links and transfers through a register.

3.4 System, TLB, and atomic operations

`mfsr rd, sysreg` reads a system register and `mtsr sysreg, rs` writes one. `rfe` returns from an exception frame. `halt` stops the virtual CPU and `break` enters the breakpoint exception path. `syscall immediate` enters the Linux syscall exception; the immediate is currently reserved for kernel diagnostics and does not replace the syscall number in `r1`.

`tlbflush` with no operand invalidates the complete translation cache. `tlbflush rN` invalidates the translation for the address in register `rN`. `cas rd, rs, 0(ra)` performs the implemented atomic compare-and-swap operation: it compares the word at the address in `ra` with the old value in `rd`, conditionally stores the replacement value from `rs`, and returns the previous memory value in `rd`. Programs requiring a portable Linux atomic API should use the libc/kernel interface rather than assuming more memory-ordering guarantees than the target documents.

4. GNU assembler and object files

The canonical GNU target is `myemulator2-elf`. Typical commands are:

```
myemulator2-elf-as -o hello.o hello.S
myemulator2-elf-ld -T toolchain/userspace/myemulator2-user.ld \
  -o hello hello.o
myemulator2-elf-objdump -dr hello
myemulator2-elf-readelf -h -l -S -r hello
```

Source uses GNU syntax. Registers are `r0`–`r15`, with `zero`, `sp`, and `lr` aliases. `#` begins a comment. Memory operands are `disp(base)`. Standard sections and symbol directives are used:

```
.section .text
.global _start
.type _start,@function
.align 2
```

The project ABI supports `.text`, `.rodata`, `.data`, `.bss`, `.section`, `.global`/`.globl`, `.local`, `.type`, `.size`, `.align`, `.balign`, `.p2align`, `.byte`, `.short`, `.word`, `.long`, `.ascii`, `.asciz`, `.space`, `.zero`, `.fill`, `.equ`, `.set`, `.include`, and ordinary GAS conditional and macro facilities where supported by the installed binutils build. Verify target-specific behavior with `myemulator2-elf-as --version` and a small assembly test; the repository's `toolchain/tests/test-binutils.py` is the regression reference.

`.word` is a 32-bit little-endian value for MyEmulator2. Use `.short` for 16-bit data and `.byte` for individual bytes. Alignment directives align the location counter; they do not change the CPU instruction width.

The project assembler recognizes `ret`, `call`, and `li` as the documented pseudo-operations. A `li` that cannot fit in one immediate is expanded into a `lui`/`ori` sequence. `call symbol` is a link-and-transfer operation. No legacy MyEmulator `push`/`pop` spelling should be assumed for this target unless it is present in the current GAS build.

5. Relocations and linking

The initial ABI uses ELF32 little-endian `RELA` relocations:

Number	Name	Use
0	`R_MYEMU_NONE`	No operation
1	`R_MYEMU_32`	32-bit data address/value
2	`R_MYEMU_BRANCH13`	Conditional branch displacement
3	`R_MYEMU_JUMP26`	`J`/`JAL` displacement
4	`R_MYEMU_HI20`	`LUI` high part
5	`R_MYEMU_LO12`	ALU-immediate low part

For a symbol `S`, addend `A`, and relocation place `P`, branch and jump relocations compute `D = S + A - (P + 4)`, require 4-byte divisibility, and encode `D/4` in the signed field. The linker rejects overflow and malformed instruction templates.

To construct an arbitrary address:

```
lui  r5, %hi(message)
ori  r5, r5, %lo(message)
```

The user linker script publishes the ELF program headers inside a loadable segment because musl startup uses `AT_PHDR`:

```
text PT_LOAD FILEHDR PHDRS FLAGS(7)
entry: _start
user image base: 0x02000000 + SIZEOF_HEADERS
```

Dynamic linking, GOT/PLT conventions, shared libraries, and dynamic TLS are not part of the stable current ABI.

6. ABI and calling convention

The stack grows downward and is 16-byte aligned at call boundaries. There is no red zone. The first four arguments are in `r1`–`r4`; later arguments are on the stack. `r1` returns a 32-bit result. A 64-bit result uses `r1:r2`, low word first.

r1–r4 and r15 are caller-saved. r5–r12 are callee-saved. sp is callee-managed. A function that executes a nested call must preserve its incoming lr.

```
.text
.global add_two
.type add_two,@function
add_two:
        add r1, r1, r2       # result in r1
        ret                  # jr r14
.size add_two, .-add_two
```

The ABI does not yet freeze aggregate argument/return rules, variadic register save areas, unwind metadata, stack canaries, floating-point register passing, or dynamic TLS. C programs should use the compiler-generated convention until those areas are formally released.

7. Linux executable startup

The canonical static executable is ELF32 little-endian with project-local machine value 0xF2E2. _start is the ELF entry point. A normal C program then enters the musl startup sequence, which prepares TLS and calls main.

Freestanding programs can provide _start directly:

```
.section .text
.global _start
.type _start,@function
_start:
        # program body
        li r1, 94            # exit_group
        li r2, 0
        syscall 0
```

The exact initial user stack and auxiliary-vector contract is still being completed in the Linux port. Programs must not infer an ARM, RISC-V, or x86 entry register convention. The supported startup path is the repository's linker script plus the current musl/minilibc crt0 implementation.

The tested crt0 pattern is:

```
.global _start
_start:
        call main
        add r2, r1, r0
        li r1, 94
        syscall 0
1:      j 1b
```

8. Linux syscall ABI

The Linux entry instruction is syscall. r1 holds the syscall number and receives the return value. Arguments occupy r2 through r7, allowing six arguments. Negative returns are Linux -errno values.

The current dispatcher is deliberately a bootstrap dispatcher, not a claim of a complete generated Linux syscall table:

Number	Name	Current implementation
17	getcwd	Yes
23	dup	Yes
24	dup3	Yes

Number	Name	Current implementation
25	`fcntl`	Yes
29	`ioctl`	Yes
34	`mkdirat`	Yes
35	`unlinkat`	Yes
38	`renameat`	Yes
49	`chdir`	Yes
56	`openat`	Yes
57	`close`	Yes
59	`pipe2`	Yes
61	`getdents64`	Yes
62	`lseek`	Yes
63	`read`	Yes
64	`write`	Yes
66	`writev`	Yes
74	`fsync`	Yes
79	`newfstatat`	Yes
80	`fstat`	Yes
93	`exit`	Yes
94	`exit_group`	Yes
96	`set_tid_address`	Yes
99	`set_robust_list`	Yes
101	`nanosleep`	Yes
124	`sched_yield`	Yes
134	`rt_sigaction`	Yes
135	`rt_sigprocmask`	Yes
160	`uname`	Yes
172	`getpid`	Yes
173	`getppid`	Yes
175	`geteuid`	Yes
178	`gettid`	Yes
214	`brk`	Yes
215	`munmap`	Yes
220	`clone`	Yes
221	`execve`	Yes
222	`mmap`	Yes
260	`wait4`	Yes
403	`clock_gettime` (32-bit time structure)	Yes
422	`futex`	Yes

`open`, `fork`, sockets, `getrandom`, `gettimeofday`, and many other calls must not be documented as available merely because their names are common on Linux. Consult `linux/arch/myemulator2/kernel/syscall.c` before using a call. The architecture syscall-number header is not yet a complete generated table.

Example direct write:

```
.section .text
.global _start
_start:
        li  r1, 64
        li  r2, 1
        lui r3, %hi(message)
        ori r3, r3, %lo(message)
        li  r4, message_end-message
        syscall 0
```

```
        li  r1, 94
        li  r2, 0
        syscall 0
.section .rodata
message: .ascii "hello from MyEmulator2\n"
message_end:
```

Reads and writes may be partial and may return negative errors. Robust code checks `r1`, retries only when the operation's documented error is interruptible, and never treats a negative result as a byte count.

9. Calling C and libc

The active hosted C library work targets musl. The project also has a small minilibc used by bootstrap tests. Static linking is the supported model. Dynamic linking and a stable shared-library ABI are not complete.

The verified libc regression exercises:

```
puts("libc-start");
malloc(32); realloc(...); calloc(...);
memset; strlen; strcmp; snprintf; printf;
uname; open; fstat; read; write; close;
mkdir; stat; nanosleep;
```

The source is `toolchain/examples/linux-libc-regression.c`. Do not infer that every POSIX or ISO C function is available from that test. Link with the project compiler driver and linker script supplied by the toolchain build, rather than invoking a host linker.

An assembly implementation of `main` follows the normal C ABI:

```
.text
.global main
.type main,@function
main:
        # r1 is the return value
        li r1, 0
        ret
.size main, .-main
```

Variadic calls, aggregate returns, and direct calls from a bare `_start` into libc require runtime initialization and are not stable ABI examples yet. Use the normal crt0 and compiler driver for libc programs.

10. Memory management and data structures

Use `lui`/`ori` to form a full pointer, then use a base register and signed displacement for members:

```
lui  r5, %hi(object)
ori  r5, r5, %lo(object)
lw   r1, 0(r5)       # first 32-bit field
lh   r2, 4(r5)       # signed 16-bit field
sb   r3, 6(r5)       # byte field
```

Keep stack frames 16-byte aligned. A four-byte local can be allocated with `addi sp, sp, -16`, stored at `0(sp)`, and released with `addi sp, sp, 16`. The compiler backend emits explicit stack updates and ordinary `lw`/`sw`; there is no architectural post-increment addressing mode.

`mmap`, `munmap`, and `brk` exist in the current Linux dispatcher, but their complete userspace memory-management contract is still tied to the evolving Linux port. Prefer libc allocation functions in hosted programs.

11. Strings, character data, and I/O

String literals use `.ascii` plus an explicit terminator or `.asciz`:

```
.section .rodata
prompt: .asciz "value: "
```

The CPU has no string instructions. Implement loops with `lb`/`lbu`, compare bytes in registers, and advance a pointer with `addi`. For terminal I/O, Linux `read` and `write` use descriptors 0, 1, and 2. Early PID 1 console startup has a kernel fallback for stdin/stdout/stderr because descriptors may not yet be inherited.

12. Debugging and disassembly

Useful tools:

```
myemulator2-elf-objdump -drwC program
myemulator2-elf-readelf -h -l -S -s -r program
myemulator2-elf-nm program
```

QEMU can run the `myemulator32` machine with a kernel and serial console:

```
qemu-system-myemulator32 -M myemulator32 -m 16M \
  -kernel .linux-build/build/vmlinux \
  -nographic -serial stdio
```

The tracked tests are the executable behavioral specification: `tests/run-myemulator32-cpu-tests.py`, `tests/run-myemulator32-integer-tests.py`, `tests/run-myemulator32-system-tests.py`, `toolchain/tests/test-isa.py` and the Linux tests under `toolchain/scripts/test-linux-*.py`.

When investigating a fault, record the fault vector, `PC`, `SR`, `SP`, `LR`, and `info` address. A user page fault is not evidence that an instruction encoding is wrong; it may be a Linux MMU, ELF, or userspace-layout failure.

13. Practical complete examples

13.1 Freestanding exit

```
.section .text
.global _start
_start:
        li r1, 94
        li r2, 0
        syscall 0
1:      j 1b
```

13.2 Function call and callee-saved register

```
.text
.global function
function:
        addi sp, sp, -16
        sw   r5, 0(sp)
        add  r5, r1, r0
        add  r1, r5, r2
        lw   r5, 0(sp)
        addi sp, sp, 16
        ret
```

13.3 Linux echo loop

`toolchain/examples/linux-echo.S` is the reference implementation. Its essential sequence is:

```
li r1, 63                 # read
li r2, 0                  # stdin
...                       # r3=buffer, r4=count
syscall 0
blt r1, r0, retry
li r1, 64                 # write
li r2, 1                  # stdout
...                       # r3=buffer, r4=bytes-read
syscall 0
```

13.4 C calling assembly

C:

```
extern int add_two(int, int);
int main(void) { return add_two(20, 22) != 42; }
```

Assembly:

```
.text
.global add_two
.type add_two,@function
add_two:
        add r1, r1, r2
        ret
.size add_two, .-add_two
```

Build using the project GCC driver and the target sysroot; do not use the host compiler to assemble this source.

14. Reference limitations and unresolved work

The following are intentionally not fabricated:

- The Linux syscall list is a maintained bootstrap dispatcher, not a complete generated architecture syscall table.
- Dynamic loader, GOT/PLT, shared-library, and dynamic-TLS formats are unfinished.
- Structure-return, aggregate argument, varargs save-area, signal-frame, and unwind ABI rules are not frozen.
- The exact initial auxiliary-vector contract is not yet a public stable ABI.
- Full libc coverage and all requested POSIX functions are not verified.
- Floating-point register calling conventions are not defined; current GCC support is soft-float.
- The legacy 16-bit MyEmulator ISA is a separate target.

These limitations are part of the manual because a technically useful programming reference must distinguish implemented behavior from design intent.

Appendix A. Authoritative source map

Subject	Source
Encoding manifest	`docs/myemulator2-encoding.json`
32-bit architecture	`docs/MYEMULATOR_2_ARCHITECTURE.md`
ABI and relocations	`docs/MYEMULATOR_2_ABI.md`
Linux machine contract	`docs/MYEMULATOR_2_LINUX_PORT.md`
QEMU decoder/translator	`qemu/target/myemulator32/translate.c`

Subject	Source
QEMU helpers/exceptions	`qemu/target/myemulator32/helper.c`
GCC instruction patterns	`toolchain/gcc/myemulator2.md`
User linker script	`toolchain/userspace/myemulator2-user.ld`
Linux syscall dispatcher	`linux/arch/myemulator2/kernel/syscall.c`
Linux entry and exception frame	`linux/arch/myemulator2/kernel/entry.S`
Assembly syscall wrappers	`toolchain/userspace/minilibc/syscall.S`
C runtime example	`toolchain/examples/linux-libc-regression.c`
Direct syscall example	`toolchain/examples/linux-echo.S`
CPU tests	`tests/run-myemulator32-*.py`
Toolchain tests	`toolchain/tests/test-*.py`

Appendix B. Revision and reproducibility

Generate this manual from the repository checkout that supplied the source map. Record the Git revision with:

```
git rev-parse HEAD
```

The PDF is a derived artifact. The Markdown source remains the reviewable authority for this document, while the implementation and executable tests remain authoritative for processor behavior.